

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1153

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 1

Total Marks:10

Annexure A

Industry Problem Solving Task

Code of Conduct:

I K Madan Mohan Reddy (192411206) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Kyreddy Madan

Annexure A

Industry Problem Solving Task

1.A mid-sized e-commerce company is building a scalable shopping platform to handle thousands of users daily. The system follows a layered object-oriented architecture, where the presentation layer provides web and mobile interfaces, the business logic layer processes orders, payments, and inventory, and the access layer communicates with an Object-Oriented Database Management System (OODBMS) to store product and customer data.

To enhance flexibility, the system uses MVC pattern for UI separation, Singleton pattern for managing database connections, and DAO pattern for data access. The design ensures that new features like discount modules can be added without modifying existing code. Performance is improved using caching and efficient query handling.

Question:

Analyze how object-oriented design principles and patterns are applied in this system, and explain how the layered architecture improves scalability and integration with the database.

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Analysis of Design Principles, Layered Architecture, and Design Patterns in a Scalable E-Commerce Shopping Platform

Introduction

A mid-sized e-commerce company requires a scalable and maintainable shopping platform capable of supporting thousands of users daily. To achieve this, the system is designed using Object-Oriented System Development (OOSD) principles, layered architecture, and design patterns. The platform consists of three main layers: Presentation Layer, Business Logic Layer, and Data Access Layer. The system stores and manages data using an Object-Oriented Database Management System (OODBMS). Design patterns such as MVC, Singleton, and DAO are used to improve flexibility, scalability, maintainability, and performance.

1. Application of Object-Oriented Design Principles

1.1 Encapsulation

Encapsulation combines data and methods within a single class and restricts direct access to internal data.

Example:

- Product class stores product information.
- Customer class stores customer details.
- Order class manages order-related operations.

Benefits:

- Improves data security.
- Prevents unauthorized access.
- Makes code easier to maintain.
-

1.2 Abstraction

Abstraction hides implementation details and provides only essential functionalities to users.

Example:

- Customers can make payments without knowing the internal payment processing mechanism.

Benefits:

- Reduces system complexity.
- Improves usability.
- Simplifies development.
-

1.3 Inheritance

Inheritance allows new classes to acquire properties and methods from existing classes.

Example:

- User (Parent Class)
 - Customer (Child Class)
 - Admin (Child Class)

Benefits:

- Promotes code reuse.
- Reduces redundancy.
- Simplifies application development.

1.4 Polymorphism

Polymorphism enables different classes to perform similar operations in different ways.

Example:

- Credit Card Payment
- UPI Payment
- Net Banking Payment

Each payment method implements payment processing differently.

Benefits:

- Increases flexibility.
- Supports multiple implementations.
- Simplifies future extensions.
-

2. Layered Architecture of the System

The e-commerce platform follows a layered architecture to separate responsibilities and improve maintainability.

A. Presentation Layer

The Presentation Layer provides the user interface through web and mobile applications.

Functions:

- Displays products and categories.
- Accepts user input.
- Shows order and payment information.
- Communicates with the Business Logic Layer.

Advantages:

- User-friendly interface.
- Easy modification of UI components.
- Independent front-end development.
-

B. Business Logic Layer

The Business Logic Layer acts as the core processing unit of the system.

Functions:

- Processes customer orders.
- Calculates discounts and taxes.
- Handles payment transactions.
- Manages inventory updates.
- Validates business rules.

Advantages:

- Centralized business operations.
- Better maintainability.
- Easy implementation of new features.
-

C. Data Access Layer

The Data Access Layer communicates directly with the OODBMS.

Functions:

- Stores product information.
- Retrieves customer details.
- Updates inventory records.
- Saves order and payment data.

Advantages:

- Separates database operations from business logic.
- Simplifies database management.
- Improves system reliability.

3. MVC (Model-View-Controller) Design Pattern

The MVC pattern separates the user interface from business processing and data management.

Model

Represents business data and application objects.

Examples:

- Product
- Customer
- Order
- Inventory

View

Represents the user interface.

Examples:

- Product Catalog Page
- Shopping Cart Page
- Checkout Page

Controller

Acts as an intermediary between Model and View.

Functions:

- Receives user requests.

- Processes inputs.
- Invokes business services.
- Returns results to the user interface.

Benefits of MVC

- Separation of concerns.
- Easier maintenance.
- Better scalability.
- Faster development process.

4. Singleton Design Pattern

The Singleton pattern ensures that only one instance of the database connection manager exists throughout the application.

Implementation

A DatabaseConnection class creates and manages a single shared database connection.

Benefits

- Reduces memory usage.
- Prevents unnecessary connection creation.
- Improves system performance.
- Ensures consistent database access.

5. DAO (Data Access Object) Design Pattern

The DAO pattern separates data access operations from business logic. Examples

- ProductDAO
- CustomerDAO
- OrderDAO
- InventoryDAO

Functions

- Insert records.
- Update records.
- Delete records.
- Retrieve records.

Benefits

- Simplifies database interaction.
- Enhances maintainability.
- Improves code organization.
- Supports database independence.

Support for Scalability and Maintainability

The system follows the Open-Closed Principle, which states that software entities should be open for extension but closed for modification.

Example: Discount Module Integration

New discount features can be added without changing existing code.

Examples:

- Seasonal Discount
- Festival Discount
- Membership Discount

These modules implement a common discount interface and can be added independently.

Benefits

- Easy feature expansion.
- Reduced risk of errors.
- Improved software maintainability.
- Better support for future business requirements.

Performance Optimization Techniques

Caching

Frequently used data is stored temporarily in memory.

Examples:

- Product Catalog
- Popular Products
- Category Lists

Benefits:

- Faster data retrieval.
- Reduced database load.
- Improved response time.

Efficient Query Handling

Optimized database queries improve overall performance.

Techniques:

- Object indexing.
- Query optimization.
- Efficient object retrieval.

Benefits:

- Faster searches.
- Better scalability.
- Reduced processing time.

6. Integration of Layers with OODBMS

The platform uses an Object-Oriented Database Management System (OODBMS) to store objects directly.

Objects Stored in OODBMS

- Product Objects
- Customer Objects
- Order Objects
- Inventory Objects
- Payment Objects

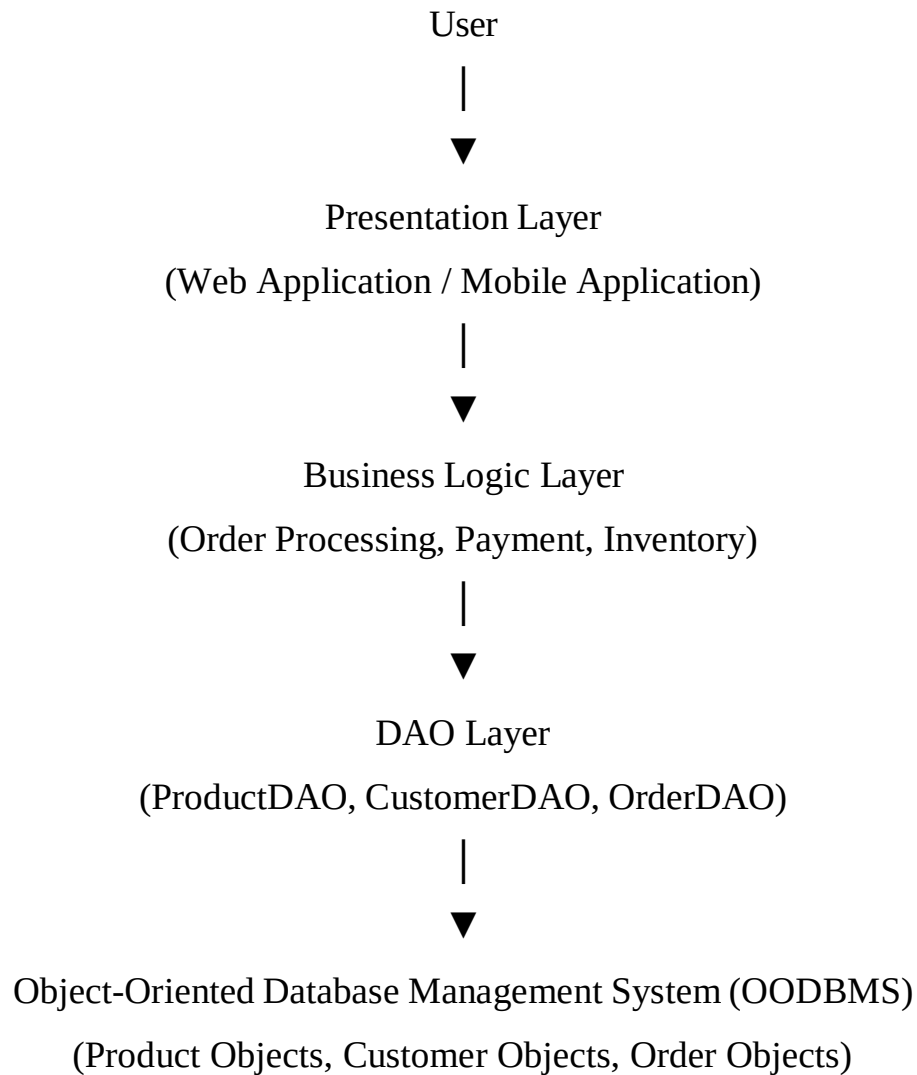
Working Process

- User interacts with the Presentation Layer.
- The request is sent to the Business Logic Layer.
- Business Logic processes the request.
- DAO classes are invoked for data operations.
- DAO communicates with the OODBMS.
- OODBMS stores or retrieves objects.
- Results are returned to the Business Logic Layer.
- The Presentation Layer displays the final output to the user.

Advantages of OODBMS

- Direct storage of objects.
- Better support for inheritance and polymorphism.
- Reduced object-relational mapping issues.
- Faster object retrieval.
- Improved development efficiency.

System Architecture Flow



Conclusion

The scalable e-commerce platform effectively applies object-oriented design principles, layered architecture, and design patterns to ensure flexibility, maintainability, and high performance. The MVC pattern separates user interface components from business logic, the Singleton pattern efficiently manages database connections, and the DAO pattern simplifies data access operations. The layered architecture provides clear separation of responsibilities, while the OODBMS enables direct object storage and retrieval.